CMSC 104 Section 2

Fall 2026

Project #3: Files, Lists and Dictionaries

Project Value: 100 points

Due Date:

Completed Project: Monday, November 23, before midnight

Overview:

The purpose of this project is to gain expertise in working with dictionaries, multi-dimensional lists, file I/O and with getting user input through the use of menus. When you have successfully completed this project you will have demonstrated mastery in the use of complex data structures in Python.

Design and constraints:

You are given many fewer constraints on the design of this project than of project 1, and thus you have much more freedom to express your own coding style. Be wary, though, because that gives you the freedom to do things wrong. Hopefully we've taught you better than that!

You must use proper design and modularity. Implement a function where you need it. If you do something multiple times, it should probably be a function. If you do something that's separate from the rest of the code, you should probably isolate it by putting it in a function. If you need to interact with the user or the system in some way, in different parts of the program, you should probably implement a helper function to take care of that.

Data:

You will be reading in three files of (fake) student records. (No real records would be used in this assignment. 😊)

The files are called ***`fall_2029.csv, spring_2030.csv and fall_2030.csv.`*** These files contain student grade records from future semesters. Each file contains two header lines,

which you will read in and then discard. After that comes some number of student records. There is one record per line. The format of each line is:

Student last name, student first name, student ID, homework 1 grade, homework 2 grade, homework 3 grade, midterm 1 grade, midterm 2 grade, and final exam grade. .

Each value is separated by a comma from the next value, which is why these files are called “Comma-separated value” or “csv” files.

IMPORTANT NOTE: You MUST read in the files using some combination of .read(), .readline(), and .readlines(). Python provides a csv.reader function and a csv.dictreader function to directly read in csv files to lists or dictionaries. Using the csv.reader or csv.dictreader function in your program will result in a minimum deduction of 10 points from your grade; possibly more. The point of this is to ensure that you learn and understand how to manipulate text files; not to learn how to take shortcuts or use tools.

Your program:

You will write a program that does the following:

Step 1:

Read in and process the three files. Each file should be put into a separate list – one list for Fall_2029, one list for Spring_2030, and one list for Fall_2030.

Each list should have one student record per element.

Each element will be a dictionary. That is, a record for a student is a dictionary; the record for a class is the list of all student dictionaries.

Each dictionary must initially have the following elements:

Key: “last name”. Value: the student last name; a string

Key: “first name”. Value: the student first name; a string

Key: “student ID” Value: the student ID; a string

Key: “h1” Value: the student’s score on homework 1; an integer

Key: “h2” Value: the student’s score on homework 2; an integer

Key: “h3” Value: the student’s score on homework 3; an integer

Key: “m1” Value: the student’s score on midterm 1; an integer

Key: “m2” Value: the student’s score on midterm 2; an integer

Key: “final” Value: the student’s score on the final exam; an integer

As a warning: some files *may* contain extra newline characters. This will result in extra rows with no data being added to the end of your list. It is your responsibility as the programmer to detect this and delete the extra rows, so that you number of students in each class is accurate.

Once you have successfully built these three lists, you may move on to the next step.

Step 2:

Insert three new key/value pairs into each dictionary in each list

Add an entry for the sum of the student’s three homework scores. You should add a Key of “homework_Total” whose value is an integer that is the sum of the three values associated with keys “h1”, “h2” and “h3.”

Add an entry for the student’s total score. You should add a Key of “Total_Score” whose value,k and integer, is the sum of all three programs and all three tests.

Now add a pair for the student’s letter grade. The Key should be “Letter_grade.” The value will be a string.

The total possible number of points is 700. To calculate the letter grade, divide the student’s total points, as calculated and stored above, by 700. If this is greater than or equal to 90%, enter an “A” for the value of “Letter_grade.”. Greater than or equal to 80% but less than 90% gets a “B”.. Greater than or equal to 70% but less than 80% gets a “C”; greater than or equal to 60% but less than 70% gets a “D”; and less than 60% gets an “F”as the value for this key. .

Once you have the three lists with their new entries correctly inserted, you may move on to the next step.

Step 3:

The three lists you have built constitute a “database” of student grades that can be queried by users who need information about student grades. Now you need to provide an interface to let users actually query your database and do their jobs.

Do this by displaying on the screen a menu of choices that the users can make. Then read in the choices that they type in, and perform the request operation.

The menu does not have to look exactly like this, but it should be very similar. In other words, you should offer the same options, in the same order, but your messages do not have to exactly match the samples.

Welcome to the Student Grade Information Service

What would you like to do?

To search by student last name, enter “L”

To search by student ID, enter “S”

To exit the system, enter “Q”

You must then read in the input the user types. All input from the keyboard is a string, so there’s no reason that your program should ever crash here, regardless of what the user types!

We’ll be a little forgiving of users here. If the user types either an uppercase “L” or a lowercase “l” move to Step 4. If the user enters either an uppercase “S” or a lowercase “s” also move to Step 4. If the user enters either an uppercase “Q” or a lowercase “q” move to Step 8, where the program terminates.

If the user enters anything other than those 6 values listed above, display an error message that says

I’m sorry, but that <echo the user input> is not a valid choice.

Replace <echo the user input> with whatever the user actually typed.

And then re-display the menu. Keep doing this until the user enters a valid value.

Step 4:

Next you must find out which semester the student to be searched for attended the class. Regardless of whether the user entered “L”, “l”, ”S”, or “s”, display the following menu:

In which semester was the student enrolled?

Enter “F” for ***fall_2029***

Enter “S” for ***spring_2030***

Enter “T” for ***fall_2030***

Enter “U” if you do not know.

Again, validate the input. This is a string, so there’s no excuse for crashing. If the user enters anything other than “F”, “f”, “S”,”s”, “T”, “t”, “U” or “u”, print the same error message you displayed in Step 3 (ooh, sounds like a helper function!!) and re-display the menu. Keep doing this until you get a valid input. When you have a valid input, you may move on to Step 5.

Step 5:

Decide what to do next. If the input from Step 3 was “L” or “l”, go to Step 6. When you call the function in Step 6, pass the value you got in Step 4.

If the input you got from Step 3 was “S” or “s”, go to Step 7. When you call the function in Step 7, pass the value you got in Step 4.

Step 6:

Here, you process a request for a student grade, search by student last name. Prompt the user to enter the last name of the student for whom the user is searching.

The user will enter a string. You will now search for this string in your database.

Your code was passed a value, read in Step 4, that indicates which semester the student was purportedly enrolled in the class. If the value from Step 4 was “F” or “f” search only in the list created from fall_2029.csv. If the value from Step 4 was “S” or “s” search only in the list created from spring_2030.csv. If the value from Step 4 was “T” or “t” search only in the list created from fall_2030.csv.. If the value was “U” or “u” you must potentially search in all 3 lists.

A match is returned regardless of case; that is, “Stark”, “STARK”, “sTaRk” and “stark” should all match the same name.

If the string entered by the user is NOT in the correct column in the list in which you are searching, (not in Any list if Step 4 provided “U” or “u’) display an error message:

Sorry, <echo user input> was not enrolled in that semester.

Please enter a student last name.

Enter “Q” to return to the main menu.

If the student presses either “Q” or “q” return to Step 3. Otherwise, stay in this loop until the user enters a student last name that is actually in the appropriate list.

If the user enters a last name that matches exactly one student in the list, display the following message:

Student Last Name Student First Name Student ID Total Points Letter Grade

<print the values from columns 0, 1, 2, 10, and 11 for the record that matched>

What If the user enters a last name that matches more than one student? E.g., does the user mean Frodo Baggins or Bungo Baggins? If the last name appears more than once in the list of dictionaries, you must ask the user to choose among the matches. Display a message similar to :

Please indicate the student for whom you are searching:

Enter 1 for: <student 1 first name> <Student 1 last name>

Enter 2 for: <student 2 first name><Student 2 last name>

Include all students with the matching last name. Here you will not worry about validating input. Presume the user will enter a valid value. Display the record as above for the selected student. That is, display

Student Last Name Student First Name Student ID Total Points Letter Grade

<print the values from columns 0, 1, 2, 10, and 11 for the record that matched>

Once you have displayed a valid record, go back to Step 3

Step 7:

Here, you process a request for a student grade, searching by student ID. This is actually easier than the last name search since an ID can match at most one student record. There is no disambiguation needed.

Prompt the user to enter the student ID of the student for whom the user is searching.

The user will enter a string. You will now search for this string in your database.

Your code was passed a value, read in Step 4, that indicates which semester the student was purportedly enrolled in the class. If the value from Step 4 was “F” or “f” search only in the list created from the Fall_2029 file. If the value from Step 4 was “S” or “s” search only in the list

created from the Spring_2030 file. For “T” or “t” search the fall_2030 file. If the value was “U” or “u” you must potentially search in all lists.

Just as with last name, a match is returned regardless of case.

If the string entered by the user is NOT in the correct column in the list in which you are searching, (not in any list if Step 4 provided “U” or “u’) display an error message:

> Sorry, <echo user input> was not enrolled in that semester.
>
> Please enter a student ID..
>
> Enter “Q” to return to the main menu.

If the student presses either “Q” or “q” return to Step 3. Otherwise, stay in this loop until the user enters a student ID that is actually in the appropriate list.

Once the user has entered a student ID that matches student in the list, display the following message:

> Student Last Name Student First Name Student ID Total Points Letter Grade
>
> <print the values from columns 0, 1, 2, 10, and 11 for the record that matched>

When you are finished displaying this message, go to Step 3.

Step 8:

Exit the program. Here, the user has chosen “Q” or “q” to exit the program. Print out a farewell message similar to the following. Again, you don’t have to use these exact words if you think you can be more poetic, but your message should convey the same idea.

> Thank you for using the interactive student grade request service.
>
> Hope to see you again soon.
>
> Shutting down now.

Grading:

The 100 points are allocated as follows:

Correct output: 30 points. Students will be given a set of 10 test cases, and the expected outputs for each. Points will be awarded for the correct response to each test case.

Program style: 70 points. Points are awarded for: proper program design (functions, function calls and return statements; proper use of dictionaries and lists; descriptive variable names, and proper use of comments.